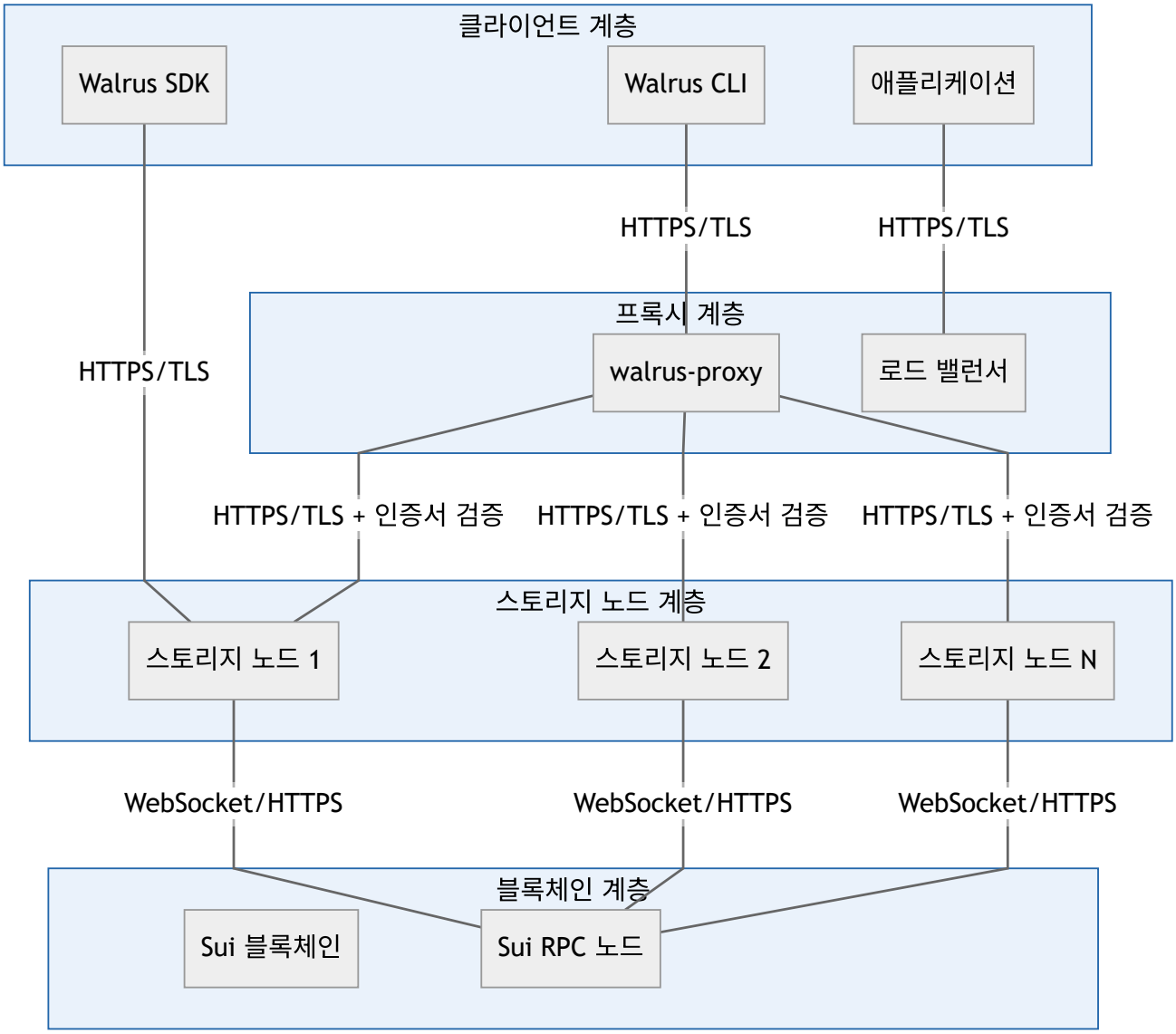
네트워크 프로토콜 및 보안 메커니즘 분석

개요

Walrus는 분산 스토리지 시스템으로서 클라이언트, 스토리지 노드, Sui 블록체인 간의 안전하고 효율적인 통신을 위해 다양한 네트워크 프로토콜과 보안 메커니즘을 구현합니다. 본 문서는 Walrus의 네트워크 통신 아키텍처, HTTP/HTTPS 기반 API, TLS 보안, 인증 메커니즘, 그리고 성능 최적화 기법들을 상세히 분석합니다.

네트워크 아키텍처 개요

통신 계층 구조



주요 통신 프로토콜

1. **HTTP/HTTPS RESTful API**: 클라이언트와 스토리지 노드 간 주요 통신
2. **TLS 1.2/1.3**: 모든 네트워크 통신의 암호화
3. **WebSocket**: Sui 블록체인 이벤트 스트리밍
4. **gRPC**: 내부 서비스 간 고성능 통신 (일부 컴포넌트)

HTTP API 프로토콜 분석

REST API 엔드포인트 구조

Walrus 스토리지 노드는 다음과 같은 RESTful API를 제공합니다:

```
// 주요 API 엔드포인트 템플릿
const METADATA_URL_TEMPLATE: &str = "/v1/blobs/:blob_id/metadata";
const SLIVER_URL_TEMPLATE: &str = "/v1/blobs/:blob_id/slivers/:sliver_pair_index
const RECOVERY_SYMBOL_URL_TEMPLATE: &str = "/v1/blobs/:blob_id/recoverySymbols/:
const BLOB_STATUS_URL_TEMPLATE: &str = "/v1/blobs/:blob_id/status";
const HEALTH_URL_TEMPLATE: &str = "/v1/health";
```

1. 블롭 메타데이터 API

메타데이터 저장

```
PUT /v1/blobs/{blob_id}/metadata
Content-Type: application/octet-stream
Authorization: Bearer <jwt_token>

Body: <BCS 인코딩된 메타데이터>
```

메타데이터 조회

```
GET /v1/blobs/{blob_id}/metadata
Accept: application/octet-stream
```

```
Response: <BCS 인코딩된 메타데이터>
```

메타데이터 상태 확인

```
GET /v1/blobs/{blob_id}/metadata/status
```

```
Response:
{
"status": "Stored" | "Nonexistent" | "Processing"
}
```

2. 슬리버 데이터 API

슬리버 저장

```
PUT /v1/blobs/{blob_id}/slivers/{pair_index}/{sliver_type}
Content-Type: application/octet-stream
Content-Length: <슬리버 크기>

Body: <BCS 인코딩된 슬리버 데이터>
```

슬리버 조회

```
GET /v1/blobs/{blob_id}/slivers/{pair_index}/{sliver_type}
```

```
Response: <BCS 인코딩된 슬리버 데이터>
```

3. 복구 심볼 API

복구 심볼 조회

```
GET /v1/blobs/{blob_id}/recoverySymbols/{symbol_id}
```

```
Response: <BCS 인코딩된 복구 심볼>
```

복구 심볼 목록 조회

```
GET /v1/blobs/{blob_id}/recoverySymbols?id={symbol_id}&targetSliver={sliver_inde
```

```
Response: [<복구 심볼 배열>]
```

4. 스토리지 확인 API

영구 블롭 확인

```
GET /v1/blobs/{blob_id}/confirmation/permanent
```

```
Response:
{
"signature": "<디지털 서명>",
```

```
"message": {
"blob_id": "<블롭 ID>",
"epoch": <에포크 번호>,
"node_id": "<노드 공개키>"
}
}
```

HTTP/2 설정 및 최적화

```rust
// HTTP/2 연결 설정
pub struct Http2Config {
pub http2_max_concurrent_streams: Option<u32>,
pub http2_max_pending_accept_reset_streams: Option<usize>,
pub http2_initial_connection_window_size: Option<u32>,
pub http2_initial_stream_window_size: Option<u32>,
pub http2_adaptive_window: bool,
}

impl Default for Http2Config {
fn default() -> Self {
Self {
http2_max_concurrent_streams: Some(1000),
http2_max_pending_accept_reset_streams: Some(100),
```

```
http2_initial_connection_window_size: Some(1024 * 1024), // 1MB

http2_initial_stream_window_size: Some(64 * 1024), // 64KB

http2_adaptive_window: true,

}

}

}
```

TLS 보안 구현

1. 자체 서명 인증서 생성

Walrus는 네트워크 공개키를 기반으로 자체 서명 인증서를 생성합니다:

```
pub fn create_unsigned_certificate(

public_key: &NetworkPublicKey,

subject_alt_name: GeneralName,

) -> CertificateDer<'static> {

let (issuer_and_subject_name, generated_alt_name) =

generate_names_from_public_key(public_key);

let alt_name_extension = SubjectAltName(vec![generated_alt_name, subject_alt_na

.to_extension(&issuer_and_subject_name, &[])

.expect("hard-coded extension is valid");

let public_key_bytes: p256::PublicKey = public_key.pubkey.into();

let tbs_certificate = TbsCertificateInner {
```

```
version: Version::V3,
serial_number,
issuer: issuer_and_subject_name.clone(),
subject: issuer_and_subject_name.clone(),
subject_public_key_info: SubjectPublicKeyInfo::from_key(public_key_bytes)
.expect("info can always be constructed from NetworkPublicKey"),
validity: Validity {
not_before: Time::GeneralTime(
GeneralizedTime::from_unix_duration(Duration::ZERO)
.expect("epoch is valid time")
),
not_after: Time::INFINITY,
},
// ... 추가 설정
};
}
```

2. 서버 이름 생성 규칙

```
pub fn server_name_from_public_key(public_key: &NetworkPublicKey) -> String {
const N_PUBLIC_KEY_BYTES_IN_SUBJECT: usize = 4;
let public_key_bytes = public_key.as_bytes();
let mut server_name = String::new();
```

```
for byte in public_key_bytes.iter().take(N_PUBLIC_KEY_BYTES_IN_SUBJECT) {
write!(&mut server_name, "{byte:x}")
.expect("write on a string always succeeds");
}
server_name.push_str(".network.walrus.alt");
server_name
}
```

3. TLS 인증서 검증

```
impl ServerCertVerifier for TlsCertificateVerifier {
fn verify_server_cert(
&self,
end_entity: &CertificateDer<'_>,
intermediates: &[CertificateDer<'_>],
server_name: &ServerName<'_>,
ocsp_response: &[u8],
now: UnixTime,
) -> Result<ServerCertVerified, rustls::Error> {
// 1. 표준 웹 PKI 검증
let verified = self.inner.verify_server_cert(
end_entity,
intermediates,
```

```
server_name,
ocsp_response,
now,
)?;
// 2. 공개키 핀닝 검증
if  self.has_pinned_public_key(end_entity) {
Ok(verified)
} else {
Err(rustls::Error::General(
"the certificate presented by the server does not match the pinned public key"
.to_owned(),
))
}
}
}
```

4. TLS 설정 옵션

```
pub  enum TlsCertificateSource {
/// PEM 인코딩된 x509 인증서와 PKCS8 개인키 로드
Pem {
certificate: PathOrInPlace<Vec<u8>>,
key: PathOrInPlace<Vec<u8>>,
```

```
},
/// 네트워크 키 쌍으로부터 자체 서명 인증서 생성
GenerateSelfSigned {
server_name: String,
network_key_pair: NetworkKeyPair,
},
}
```

```
#[derive(Debug, Default, Clone, Deserialize, Serialize, PartialEq)]
pub struct TlsConfig {
/// REST API에서 TLS 비활성화
/// 네트워크의 다른 서비스에서 TLS 암호화를 처리하는 경우에만 비활성화
pub disable_tls: bool,
/// PEM 인코딩된 x509 인증서 경로
pub certificate_path: Option<PathBuf>,
}
```

인증 및 권한 관리

1. 공개키 기반 노드 인증

```
impl StorageNodeClient {
pub fn for_storage_node(
```

```
address: &str,
public_key: &NetworkPublicKey,
) -> Result<StorageNodeClient, ClientBuildError> {
Self::builder()
.authenticate_with_public_key(public_key.clone())
.build(address)
}
}
```

2. 디지털 서명 검증

```
pub struct AuthenticatedSliver {
pub sliver: Sliver,
pub merkle_proof: MerkleProof,
pub signature: Signature,
}

impl AuthenticatedSliver {
pub fn verify(&self, public_key: &PublicKey, merkle_root: &Hash) -> Result<(),
// 1. 머클 증명 검증
self.merkle_proof.verify(&self.sliver.hash(), merkle_root)?;
// 2. 디지털 서명 검증
public_key.verify(&self.sliver.to_bytes(), &self.signature)?;
```

```
Ok(())
}
}
```

3. JWT 기반 인증 (선택적)

```
pub struct AuthToken {
pub node_id: NodeId,
pub permissions: Vec<Permission>,
pub expires_at: SystemTime,
}

impl AuthToken {
pub fn verify(&self, secret: &[u8]) -> Result<(), AuthError> {
if self.expires_at < SystemTime::now() {
return Err(AuthError::TokenExpired);
}
// JWT 서명 검증
jsonwebtoken::decode::<Claims>(
&self.token,
&DecodingKey::from_secret(secret),
&Validation::default()
)?;
```

```
Ok(())
}
}
```

네트워크 최적화 기법

1. 연결 풀링

```
pub struct ConnectionPool {
pools: HashMap<NodeId, Pool<HttpConnection>>,
max_connections: usize,
connection_timeout: Duration,
idle_timeout: Duration,
}

impl ConnectionPool {
pub async fn get_connection(&self, node_id: NodeId) -> Result<PooledConnectio
let pool = self.pools.get(&node_id)
.ok_or(NetworkError::NodeNotFound)?;
// 연결 풀에서 재사용 가능한 연결 획득
let connection = pool.get().await
.map_err(NetworkError::from)?;
Ok(connection)
```

```
}
pub fn configure_pool(&mut self, node_id: NodeId) {
let pool = Pool::builder()
.max_size(self.max_connections)
.connection_timeout(self.connection_timeout)
.idle_timeout(Some(self.idle_timeout))
.build();
self.pools.insert(node_id, pool);
}
}
```

2. 배치 요청 처리

```
pub struct BatchRequest {
pub requests: Vec<SliverRequest>,
pub timeout: Duration,
pub max_concurrent: usize,
}

impl StorageNodeClient {
pub async fn batch_store_slivers(
&self,
```

```
batch: BatchRequest
) -> Result<Vec<StoreResult>, NetworkError> {
let semaphore = Arc::new(Semaphore::new(batch.max_concurrent));
let futures: Vec<_> = batch.requests
.into_iter()
.map(|req| {
let semaphore = semaphore.clone();
async move {
let _permit = semaphore.acquire().await?;
self.store_sliver_single(req).await
}
})
.collect();
timeout(batch.timeout, join_all(futures)).await?
}
}
```

3. 압축 및 직렬화 최적화

```
pub struct CompressedTransport {
compression_level: u32,
compression_threshold: usize,
}
```

```
impl CompressedTransport {
pub fn compress_payload(&self, data: &[u8]) -> Result<Vec<u8>, CompressionErro
if data.len() < self.compression_threshold {
return Ok(data.to_vec());
}
let mut encoder = snap::raw::Encoder::new();
encoder.compress_vec(data)
.map_err(CompressionError::from)
}
pub fn decompress_payload(&self, compressed: &[u8]) -> Result<Vec<u8>, Compres
let mut decoder = snap::raw::Decoder::new();
decoder.decompress_vec(compressed)
.map_err(CompressionError::from)
}
}
```

4. 요청 제한 및 백프레셔

```
pub struct RateLimiter {
recovery_symbols_limit: Option<Arc<Semaphore>>,
request_rate_limit: Arc<Semaphore>,
burst_capacity: usize,
```

```
}


impl RateLimiter {
pub  async  fn  acquire_recovery_permit(&self) -> Result<SemaphorePermit, RateLi
if  let Some(ref  limiter) = self.recovery_symbols_limit {
limiter.acquire().await
.map_err(|_| RateLimitError::RecoverySymbolsLimitExceeded)
} else {
// 제한 없음
Ok(SemaphorePermit::dummy())
}
}
pub  async  fn  acquire_request_permit(&self) -> Result<SemaphorePermit, RateLim
self.request_rate_limit.acquire().await
.map_err(|_| RateLimitError::RequestRateLimitExceeded)
}
}
```

에러 처리 및 복구

1. 네트워크 에러 분류

```
#[derive(Debug, thiserror::Error)]
```

```rust
pub enum NetworkError {
#[error("연결 시간 초과")]
ConnectionTimeout,
#[error("TLS 핸드셰이크 실패: {0}")]
TlsHandshakeFailure(String),
#[error("인증서 검증 실패: {0}")]
CertificateVerificationFailure(String),
#[error("HTTP 요청 실패: {status_code}")]
HttpRequestFailure { status_code: u16 },
#[error("직렬화/역직렬화 오류: {0}")]
SerializationError(String),
#[error("네트워크 연결 불가: {0}")]
NetworkUnreachable(String),
}
```

2. 재시도 메커니즘

```rust
pub struct RetryConfig {
pub max_retries: usize,
pub initial_delay: Duration,
pub max_delay: Duration,
pub backoff_multiplier: f64,
}
```

```rust
impl RetryConfig {
pub  async  fn  execute_with_retry<F, T, E>(&self, mut  operation: F) -> Result<
where
F: FnMut() -> Pin<Box<dyn Future<Output = Result<T, E>> + Send>>,
E: std::fmt::Debug,
{
let  mut  delay = self.initial_delay;
for  attempt  in  0..=self.max_retries {
match  operation().await {
Ok(result) => return Ok(result),
Err(error) if  attempt == self.max_retries => return Err(error),
Err(error) => {
tracing::warn!(?error, attempt, "작업 실패, 재시도 중");
tokio::time::sleep(delay).await;
delay = std::cmp::min(
Duration::from_millis(
(delay.as_millis() as f64 * self.backoff_multiplier) as u64
),
self.max_delay,
);
}
}
```

```
}
unreachable!()
}
}
```

3. 회로 차단기 패턴

```
pub struct CircuitBreaker {
failure_threshold: usize,
recovery_timeout: Duration,
state: Arc<Mutex<CircuitState>>,
}

#[derive(Debug)]
enum CircuitState {
Closed { failure_count: usize },
Open { opened_at: Instant },
HalfOpen,
}

impl CircuitBreaker {
pub async fn call<F, T, E>(&self, operation: F) -> Result<T, CircuitBreakerEr
where
```

```
F: Future<Output = Result<T, E>>,

{

let  mut  state = self.state.lock().await;

match *state {

CircuitState::Open { opened_at } => {

if  opened_at.elapsed() > self.recovery_timeout {

*state = CircuitState::HalfOpen;

} else {

return Err(CircuitBreakerError::CircuitOpen);

}

}

CircuitState::Closed { failure_count } if  failure_count >= self.failure_thresho

*state = CircuitState::Open { opened_at: Instant::now() };

return Err(CircuitBreakerError::CircuitOpen);

}

_ => {}

}

drop(state);

match  operation.await {

Ok(result) => {

let  mut  state = self.state.lock().await;

*state = CircuitState::Closed { failure_count: 0 };

Ok(result)
```

```
}
Err(error) => {
let mut state = self.state.lock().await;
match *state {
CircuitState::Closed { failure_count } => {
*state = CircuitState::Closed { failure_count: failure_count + 1 };
}
CircuitState::HalfOpen => {
*state = CircuitState::Open { opened_at: Instant::now() };
}
_ => {}
}
Err(CircuitBreakerError::OperationFailed(error))
}
}
}
}
```

메트릭 및 모니터링

1. HTTP 요청 메트릭

```
lazy_static! {
```

```
static ref HTTP_REQUESTS_TOTAL: CounterVec = register_counter_vec!(
"walrus_http_requests_total",
"HTTP 요청 총 개수",
&["method", "endpoint", "status_code"]
).unwrap();
static ref HTTP_REQUEST_DURATION: HistogramVec = register_histogram_vec!(
"walrus_http_request_duration_seconds",
"HTTP 요청 처리 시간",
&["method", "endpoint"],
vec![0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0]
).unwrap();
static ref TLS_HANDSHAKE_DURATION: Histogram = register_histogram!(
"walrus_tls_handshake_duration_seconds",
"TLS 핸드셰이크 소요 시간",
vec![0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0]
).unwrap();
}


pub fn record_http_request(method: &str, endpoint: &str, status_code: u16, dur
HTTP_REQUESTS_TOTAL
.with_label_values(&[method, endpoint, &status_code.to_string()])
.inc();
HTTP_REQUEST_DURATION
```

```
.with_label_values(&[method, endpoint])
.observe(duration.as_secs_f64());
}
```

2. 연결 상태 모니터링

```
lazy_static! {
static ref ACTIVE_CONNECTIONS: GaugeVec = register_gauge_vec!(
"walrus_active_connections",
"활성 연결 수",
&["node_id", "connection_type"]
).unwrap();
static ref CONNECTION_ERRORS: CounterVec = register_counter_vec!(
"walrus_connection_errors_total",
"연결 오류 총 개수",
&["node_id", "error_type"]
).unwrap();
}

pub fn track_connection_metrics(node_id: &str, connection_type: &str, active:
let gauge = ACTIVE_CONNECTIONS.with_label_values(&[node_id, connection_type]);
if active {
```

```
gauge.inc();
} else {
gauge.dec();
}
}

pub fn record_connection_error(node_id: &str, error_type: &str) {
CONNECTION_ERRORS
.with_label_values(&[node_id, error_type])
.inc();
}
```

보안 모범 사례

1. 인증서 관리

- **자동 갱신**: Let's Encrypt 등을 통한 인증서 자동 갱신
- **키 순환**: 정기적인 네트워크 키 쌍 교체
- **인증서 핀닝**: 공개키 핀닝을 통한 중간자 공격 방지

2. 네트워크 보안

- **방화벽 설정**: 필요한 포트만 개방
- **DDoS 보호**: 요청 제한 및 백프레셔 메커니즘
- **감사 로깅**: 모든 인증 및 권한 부여 이벤트 로깅

3. 데이터 보호

- **전송 중 암호화**: 모든 네트워크 통신에 TLS 적용
- **무결성 검증**: 디지털 서명을 통한 데이터 무결성 보장
- **접근 제어**: 역할 기반 접근 제어 (RBAC) 구현

성능 튜닝 가이드

1. 네트워크 설정 최적화

```yaml
# 네트워크 성능 최적화 설정
rest_server:
http2_config:
http2_max_concurrent_streams: 1000
http2_initial_connection_window_size: 1048576  # 1MB
http2_initial_stream_window_size: 65536  # 64KB
http2_adaptive_window: true

# 연결 풀 설정
connection_pool:
max_connections_per_node: 100
connection_timeout_secs: 30
idle_timeout_secs: 300
```

2. 압축 설정

```yaml
# 압축 설정
compression:
enabled: true
level: 6  # 1(빠름) ~ 9(높은 압축률)
threshold_bytes: 1024  # 1KB 이상만 압축
```

3. 요청 제한 설정

```yaml
# 요청 제한 설정
rate_limiting:
max_requests_per_second: 1000
burst_capacity: 2000
recovery_symbols_limit: 100
```

문제 해결 가이드

1. 일반적인 네트워크 문제

TLS 핸드셰이크 실패

```bash
# 인증서 확인
openssl  s_client  -connect <node_address>:443  -servername <server_name>
```

```bash
# 로그 확인
```

```
grep "TLS handshake" /var/log/walrus/node.log
```

연결 시간 초과

```
# 네트워크 연결성 확인
telnet <node_address> <port>

# 방화벽 규칙 확인
iptables -L -n
```

2. 성능 문제 진단

높은 지연 시간

```
# HTTP 요청 지연 시간 확인
curl -w "@curl-format.txt" -o /dev/null -s "https://<node_address>/v1/heal

# 네트워크 지연 시간 측정
ping <node_address>
traceroute <node_address>
```

연결 풀 고갈

```
# 활성 연결 수 확인
```

```
ss -tuln | grep <port>


# 메트릭 확인
curl http://<node_address>:9184/metrics | grep walrus_active_connections
```

결론

Walrus의 네트워크 프로토콜 및 보안 메커니즘은 분산 스토리지 시스템의 요구사항을 충족하기 위해 다층적으로 설계되었습니다. HTTP/HTTPS 기반의 RESTful API, TLS를 통한 암호화, 공개키 기반 인증, 그리고 다양한 성능 최적화 기법들이 통합되어 안전하고 효율적인 네트워크 통신을 제공합니다.

주요 특징:

- **보안**: TLS 1.2/1.3, 공개키 핀닝, 디지털 서명 검증
- **성능**: HTTP/2, 연결 풀링, 배치 처리, 압축
- **신뢰성**: 재시도 메커니즘, 회로 차단기, 에러 처리
- **관찰성**: 포괄적인 메트릭 수집 및 모니터링

이러한 네트워크 인프라는 Walrus가 대규모 분산 환경에서 안정적으로 운영될 수 있는 기반을 제공합니다.